In this assessment, students will learn about using Jenkins. The goal is to understand general concepts and behaviors related to Jenkins as a build server.

1. Installing Jenkins

To install Jenkins using a Docker container the student can do the following command:

```
docker run -p 8080:8080 -p 50000:50000 -v /var/jenkins_home:/var/jenkins_home --
name jenkins -d jenkins/jenkins:lts
```

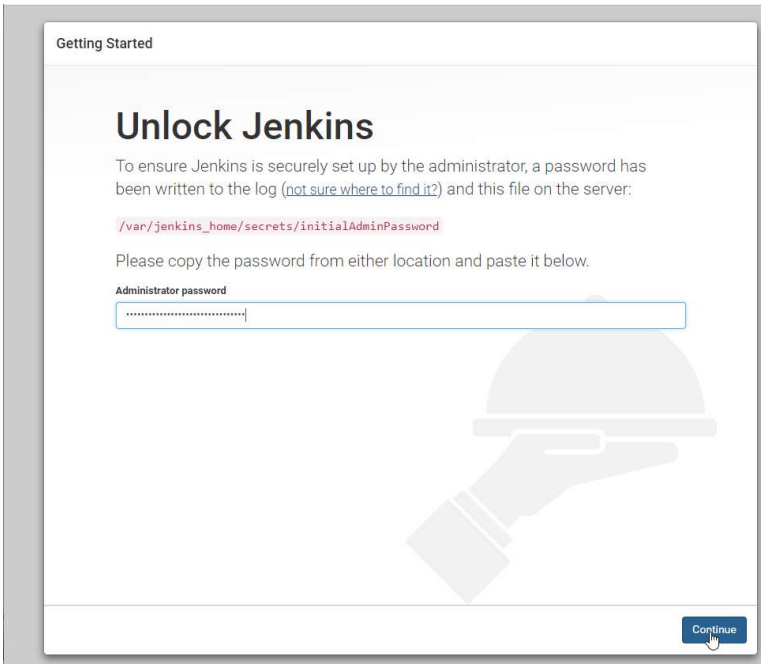
Note: if the containers don't start because of directory permissions, you can execute on the path /var/jenkins_home the command: "chmod 777 /var/jenkins_home"

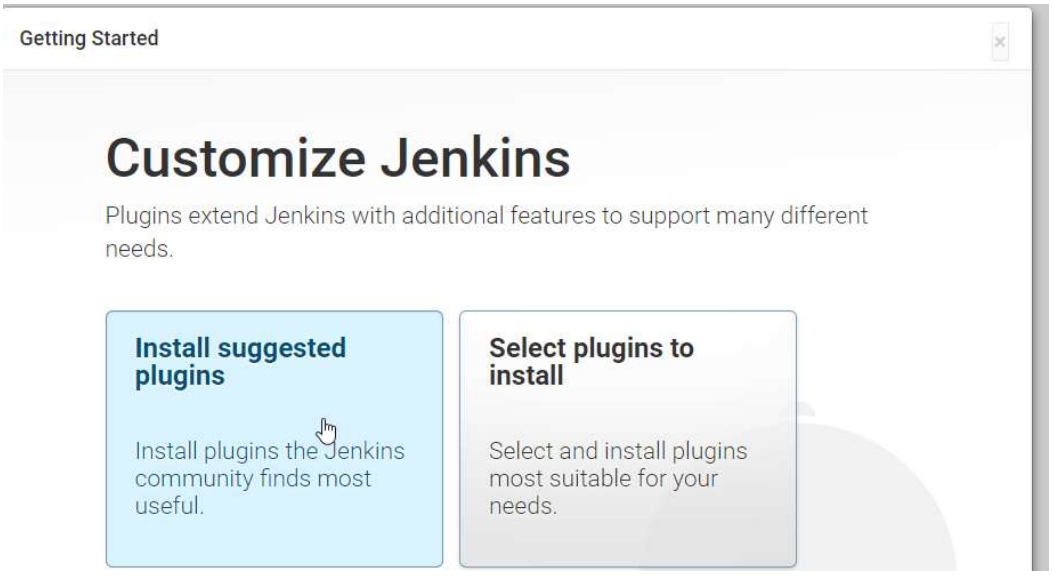
To more information, please consult the official website: https://hub.docker.com/r/jenkins/jenkins

When you start the container, the Frontend will ask for the administration password.
You can get the password using one of the following methods:

- Accessing the file: `/var/jenkins_home/secrets/initialAdminPassword`
- Accessing the container logs

Afterwards, you can follow the steps described on the following images to create the initial setup (when accessing the port 8080 on localhost):





Jenkins might take some time to install the suggested plugins.

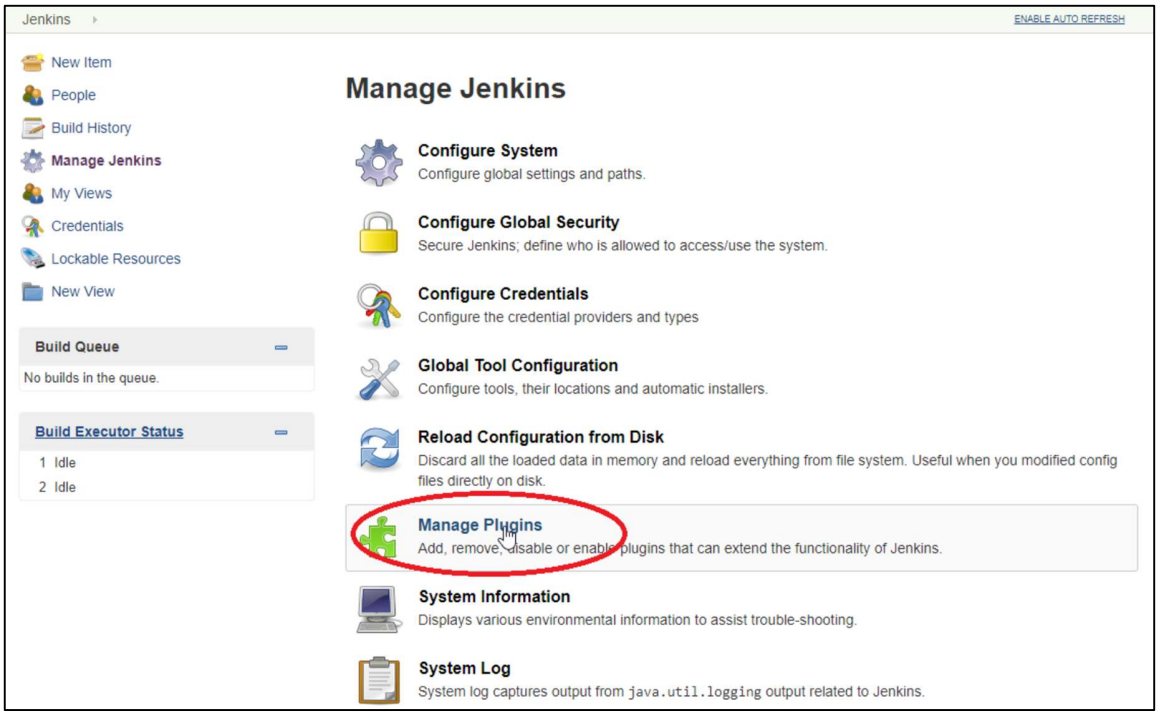
2. Creating a first Job: UI Jobs – Setup Jenkins

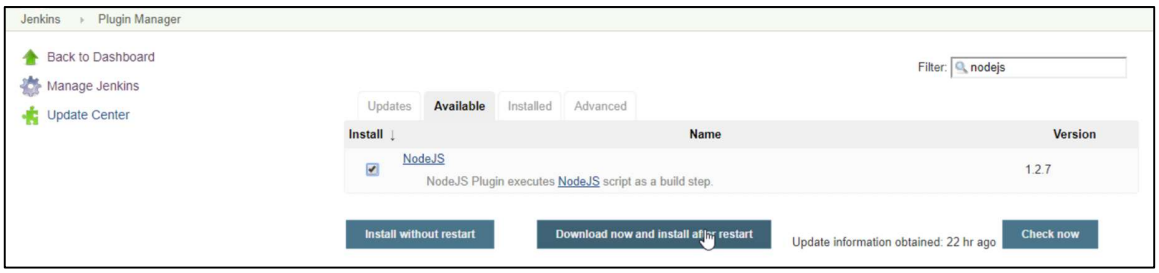
We are going to create a new project based on NodeJS - a “Hello World” application.

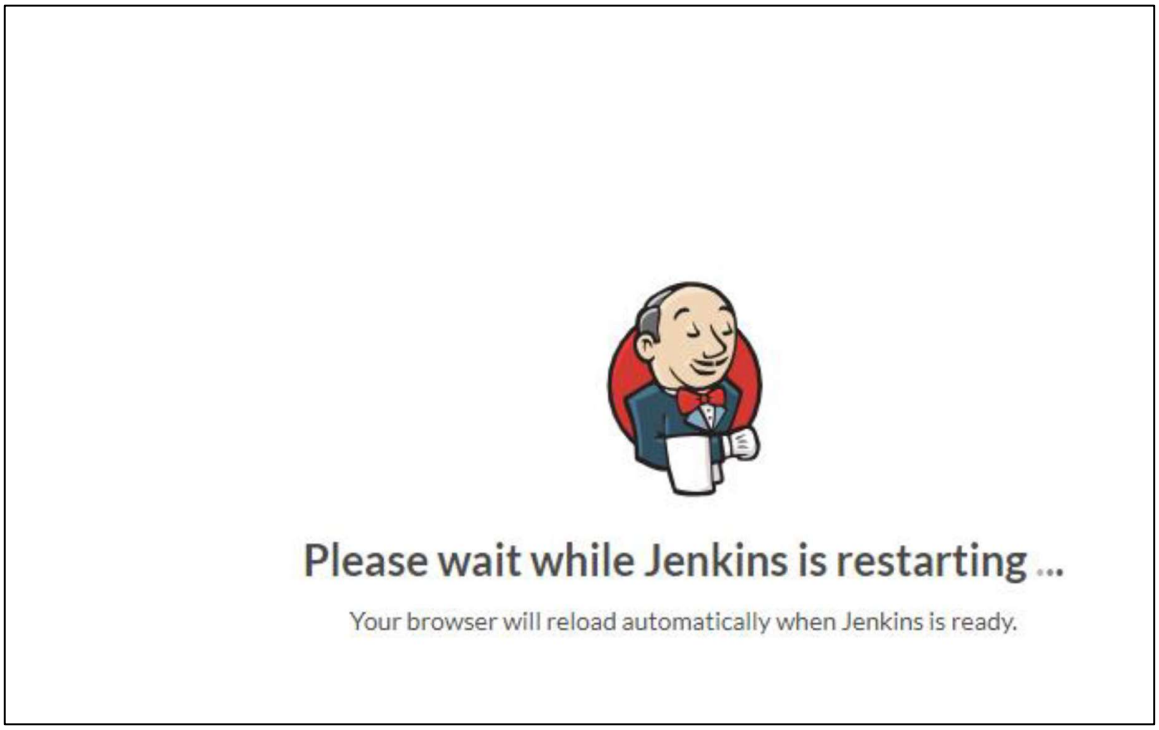
Note: Jenkins might ask if you want to restart. Since you are in a container, if you restart the container will go down. Hence, you need to start again, using the command “docker container start <id>. Another approach is to not restart Jenkins and continue to work normally.

First step is to install NodeJS plugin:

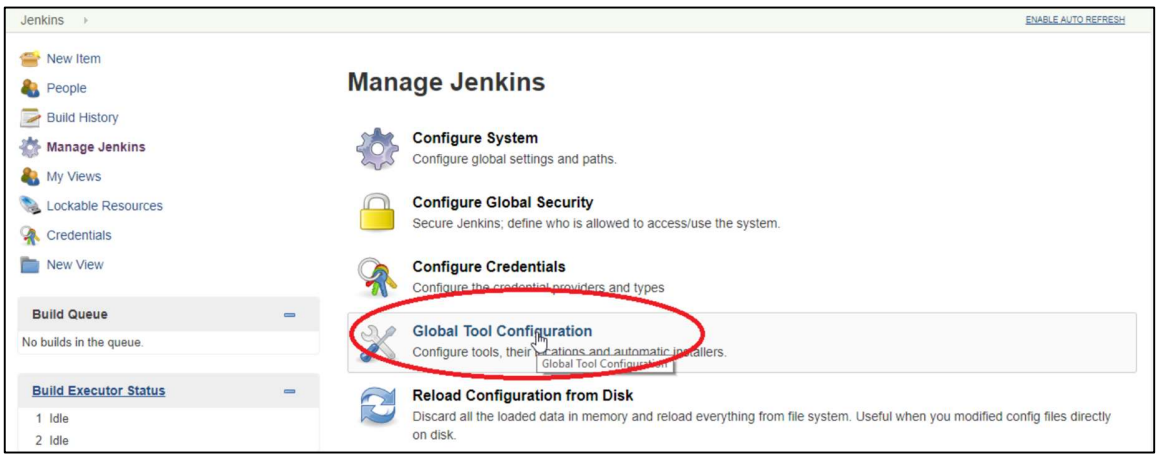




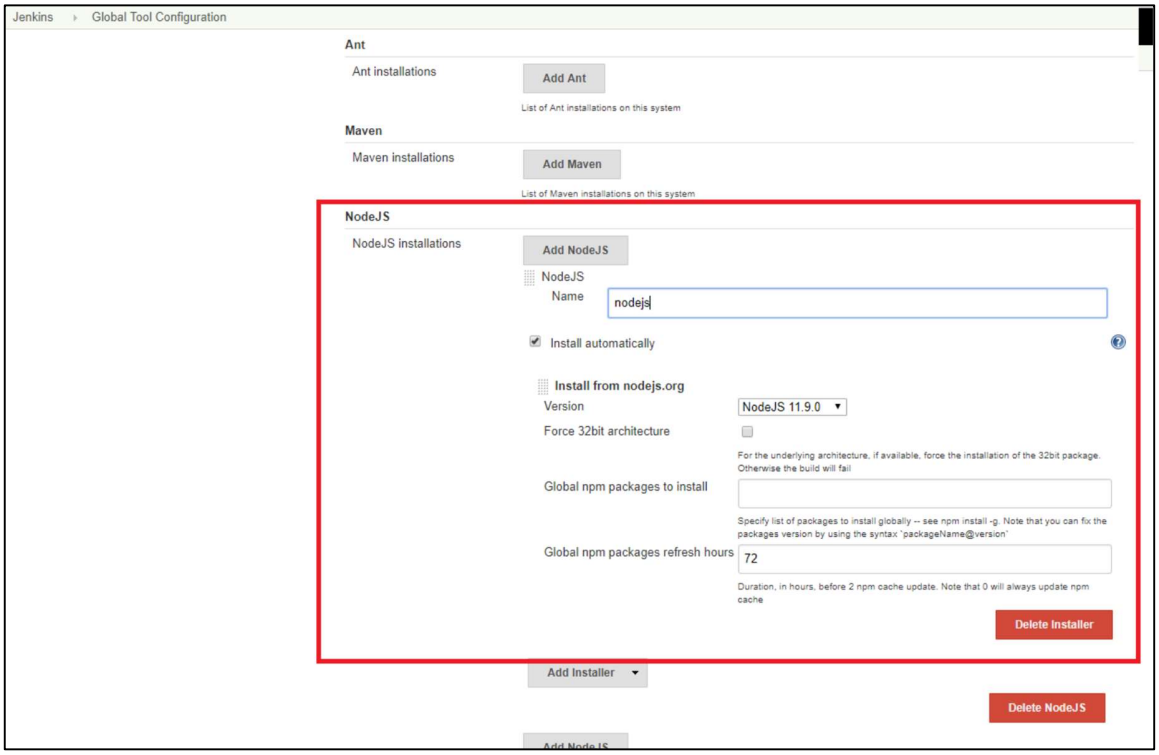




Second step is to configure the NodeJS installation:

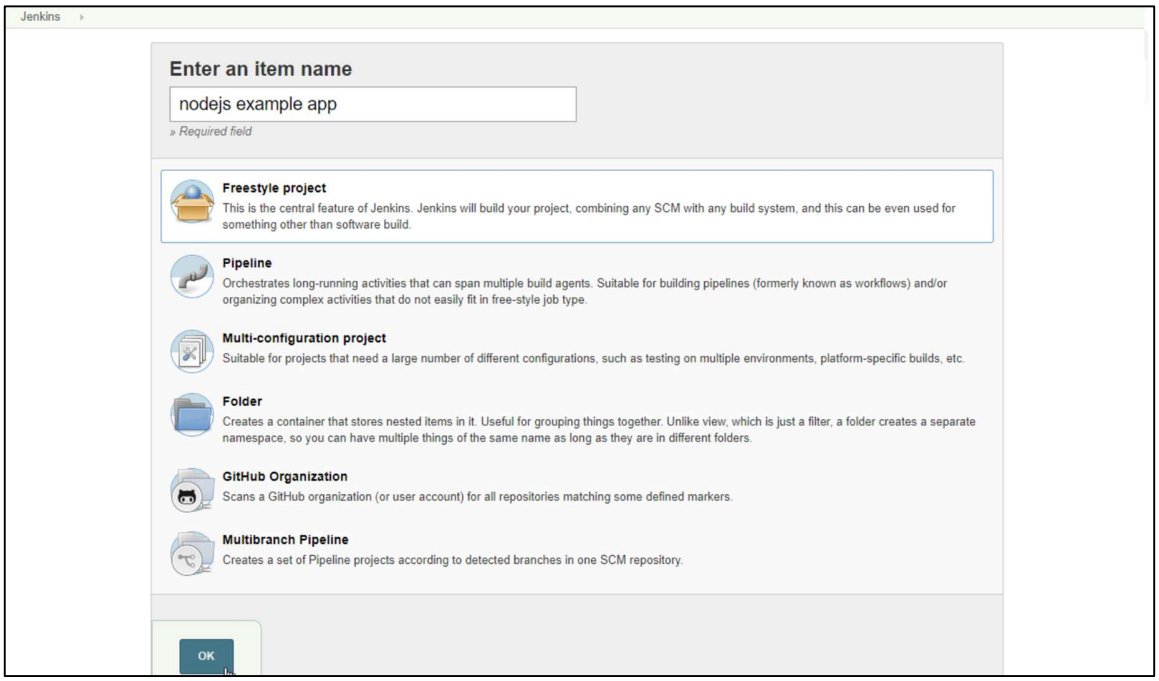


In the following image, it was given the name “nodejs” to the installation. This name will be used on the Jobs setup.



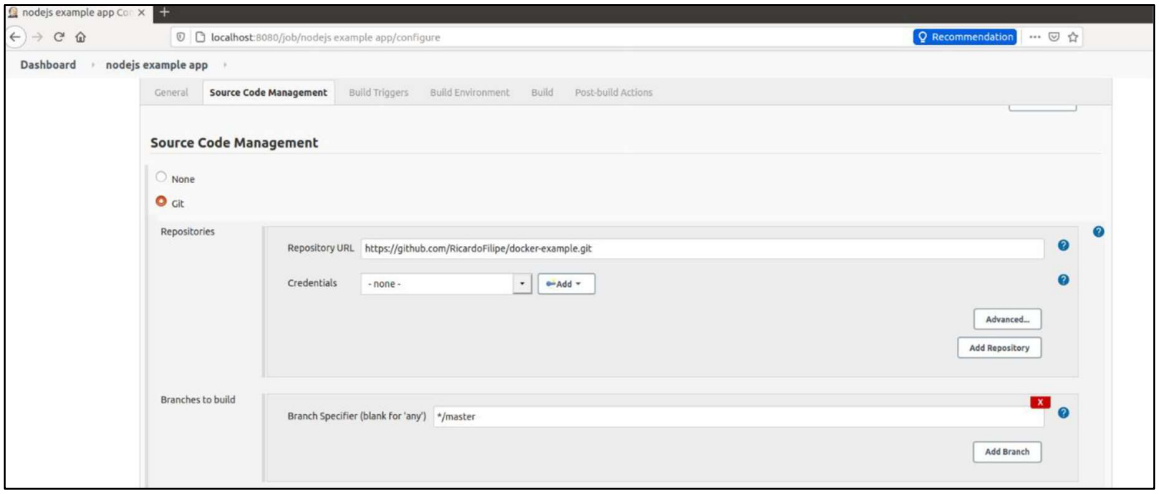
3. Creating a first Job: UI Jobs – Job Jenkins

The student can create the Job following the flow described in the images below:

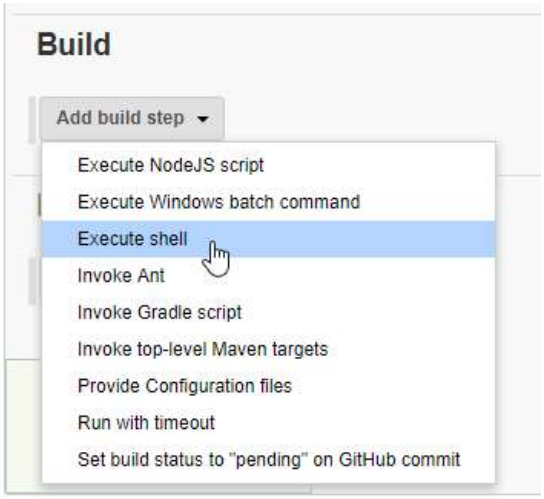


Afterwards, Jenkins will ask about the Git repository URL.
You can use the URL https://github.com/RicardoFilipe/jenkins-pd-nodejs.git . This repository includes a sample project.

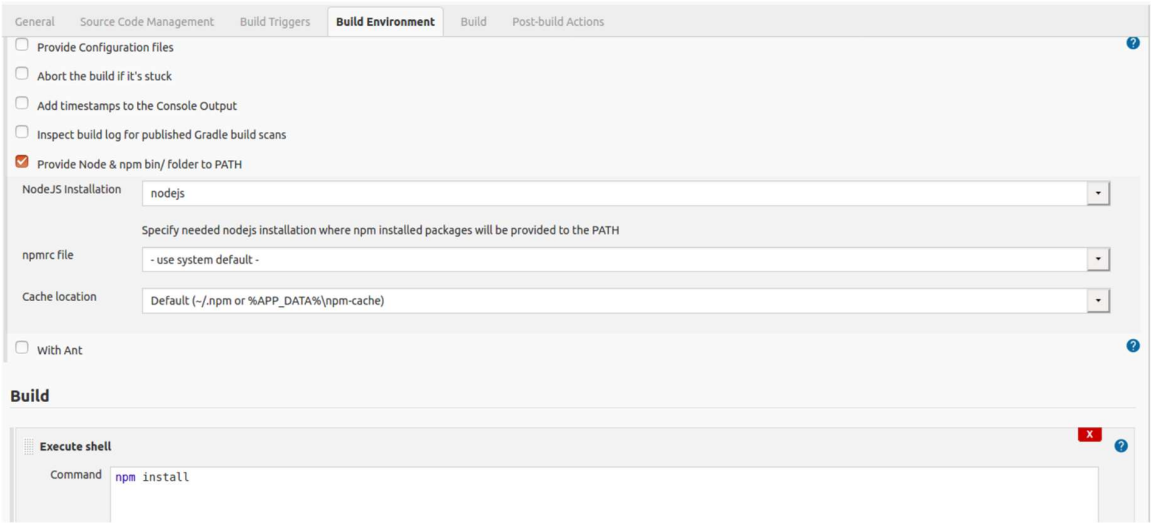
Note: The following image has the branch master, but it should be used the main branch.



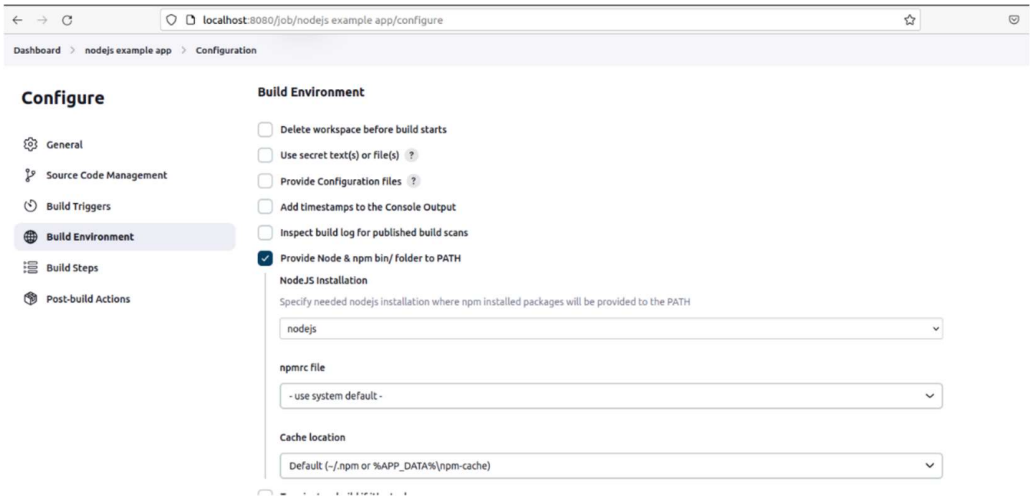
Then you can add the shell:



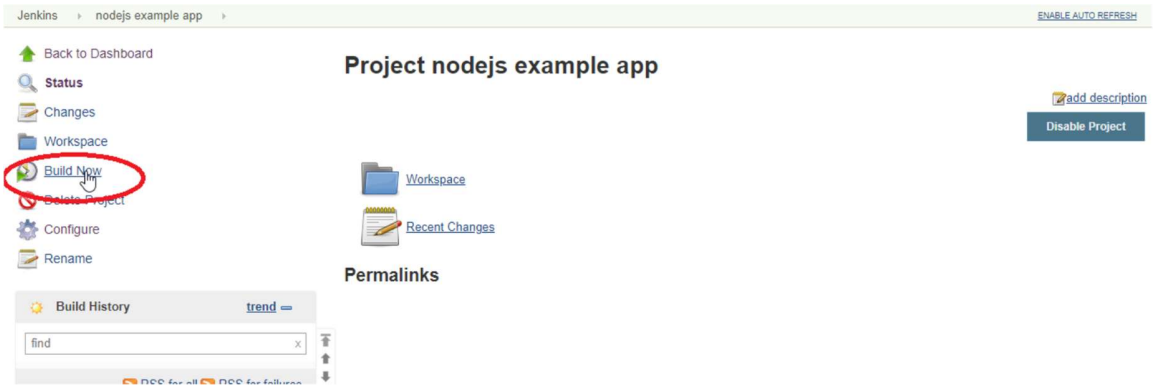
And execute the command “npm install”. Take into consideration the alias for the “NodeJS Installation”:

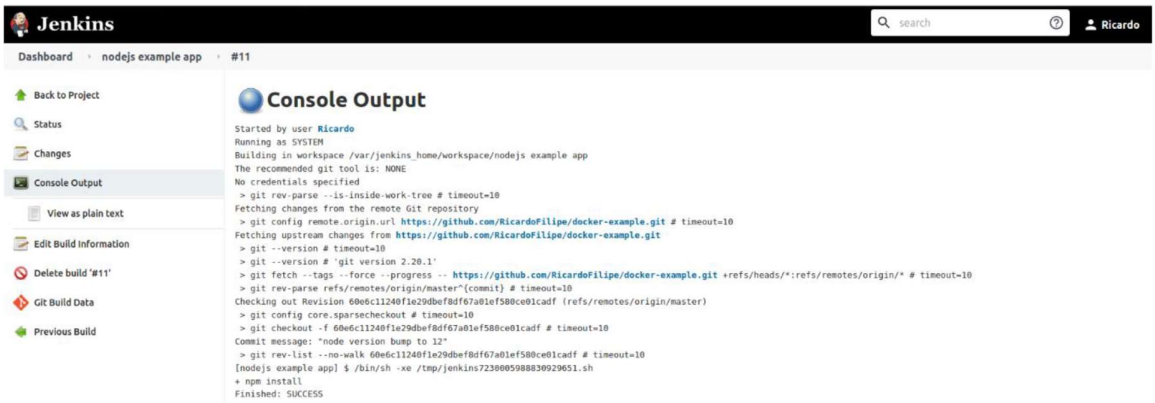


Note: Jenkins FE look may change depending of the version. E.g.



Afterwards, make build and check that the job returned success:



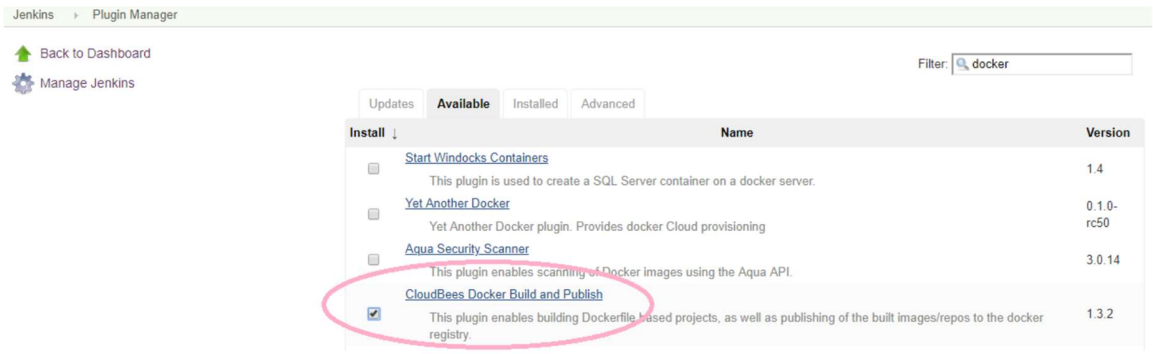


The App can be found in the following place:

```
jenkins@fa95b9b52196:~/workspace/nodejs example app$ pwd
/var/jenkins_home/workspace/nodejs example app
jenkins@fa95b9b52196:~/workspace/nodejs example app$ ls -l
total 60
-rw-r--r--  1 jenkins jenkins   356 Feb 20 18:59 docker-compose.yml
-rw-r--r--  1 jenkins jenkins    79 Feb 20 18:59 Dockerfile
-rw-r--r--  1 jenkins jenkins  1055 Feb 20 18:59 index-db.js
-rw-r--r--  1 jenkins jenkins   316 Feb 20 18:59 index.js
drwxr-xr-x  2 jenkins jenkins  4096 Feb 20 18:59 misc
drwxr-xr-x 80 jenkins jenkins  4096 Mar  7 16:00 node_modules
-rw-r--r--  1 jenkins jenkins   397 Feb 20 19:06 package.json
-rw-r--r--  1 jenkins jenkins 26116 Feb 20 19:06 package-lock.json
drwxr-xr-x  2 jenkins jenkins  4096 Feb 20 18:59 test
jenkins@fa95b9b52196:~/workspace/nodejs example app$
```

4. Creating a Docker image using Jenkins Job

Install the Docker plugin:



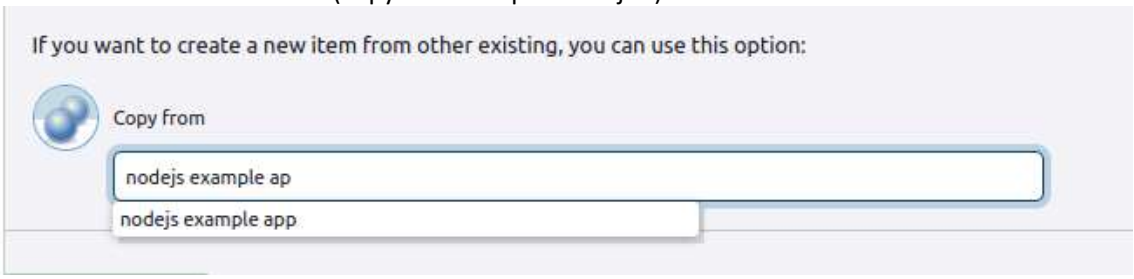
The previous image from Jenkins will **not** work with the plugin of Docker. We need some slight changes from the official Jenkins Image. You can consult the Dockerfile from image ricardofilipe/jenkins-pd:lts in here. Try to create the image on your own VM and push to your dockerhub repository.

Otherwise, you can use the new customized image, and the previous volume, using the following command:

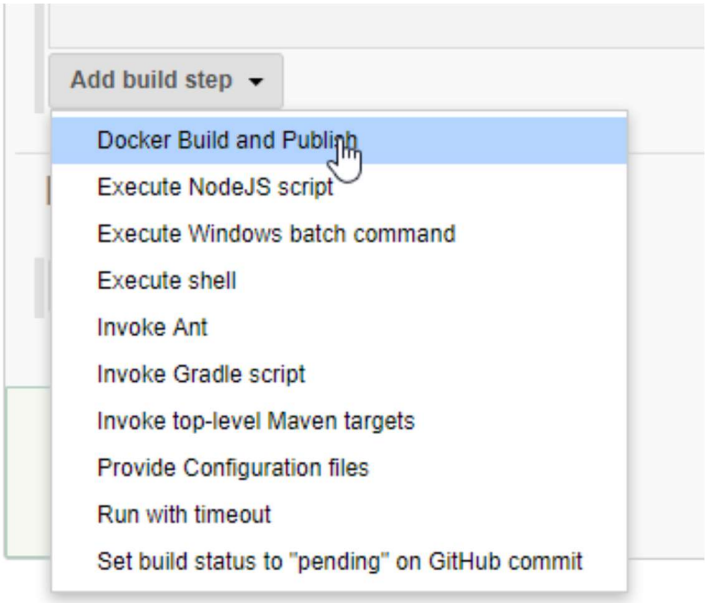
```
docker run -p 8080:8080 -p 50000:50000 -v /var/jenkins_home:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock --name jenkins  --rm ricardofilipe/jenkins-pd:lts
```

4.1 – The student is using a new container and the previous volume. Is there any information inside the container missing from previous exercises?

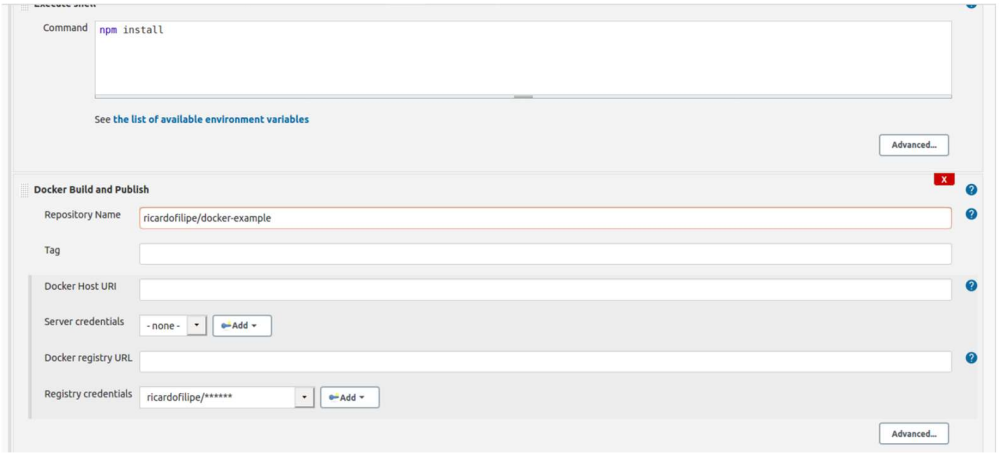
4.2- Create a new Jenkins Job (copy from the previous job):



Associate a new step to the job:



On the Repository Name Placeholder insert/create a new alias to be able to connect to the student dockerhub account. I.e. press "Add" near the Repository credentials and type your username/password of Dockerhub (using the credential with username and password option). For the "alias" field you can choose "dockerhub" that will be used in the next exercises.

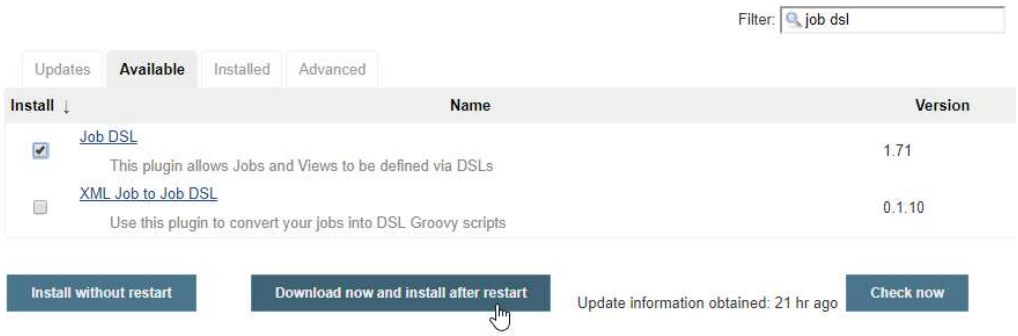


Check the following items:

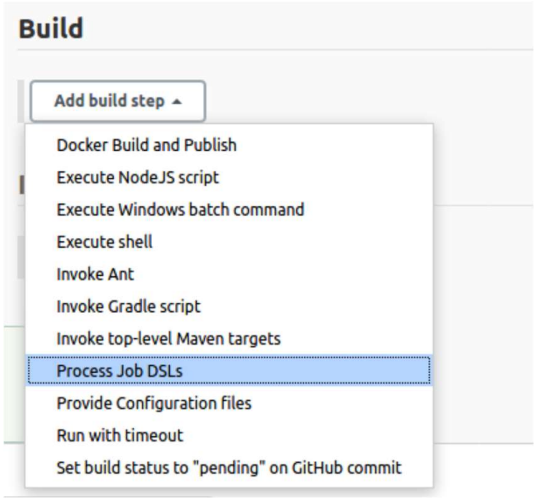
- The job returned success
- The newly created image is on dockerhub
- To test the application you can pull the new image and run the container (change the image reference from the below command to your own repository):
    - Docker run -p 3000:3000 ricardofilipe/docke-example:latest
    - The result will be "Hello World" on the browser

5. Creating a DSL Hello World

We are going to create a DSL job of type “Hello World”
First, we are going to install the DSL plugin:



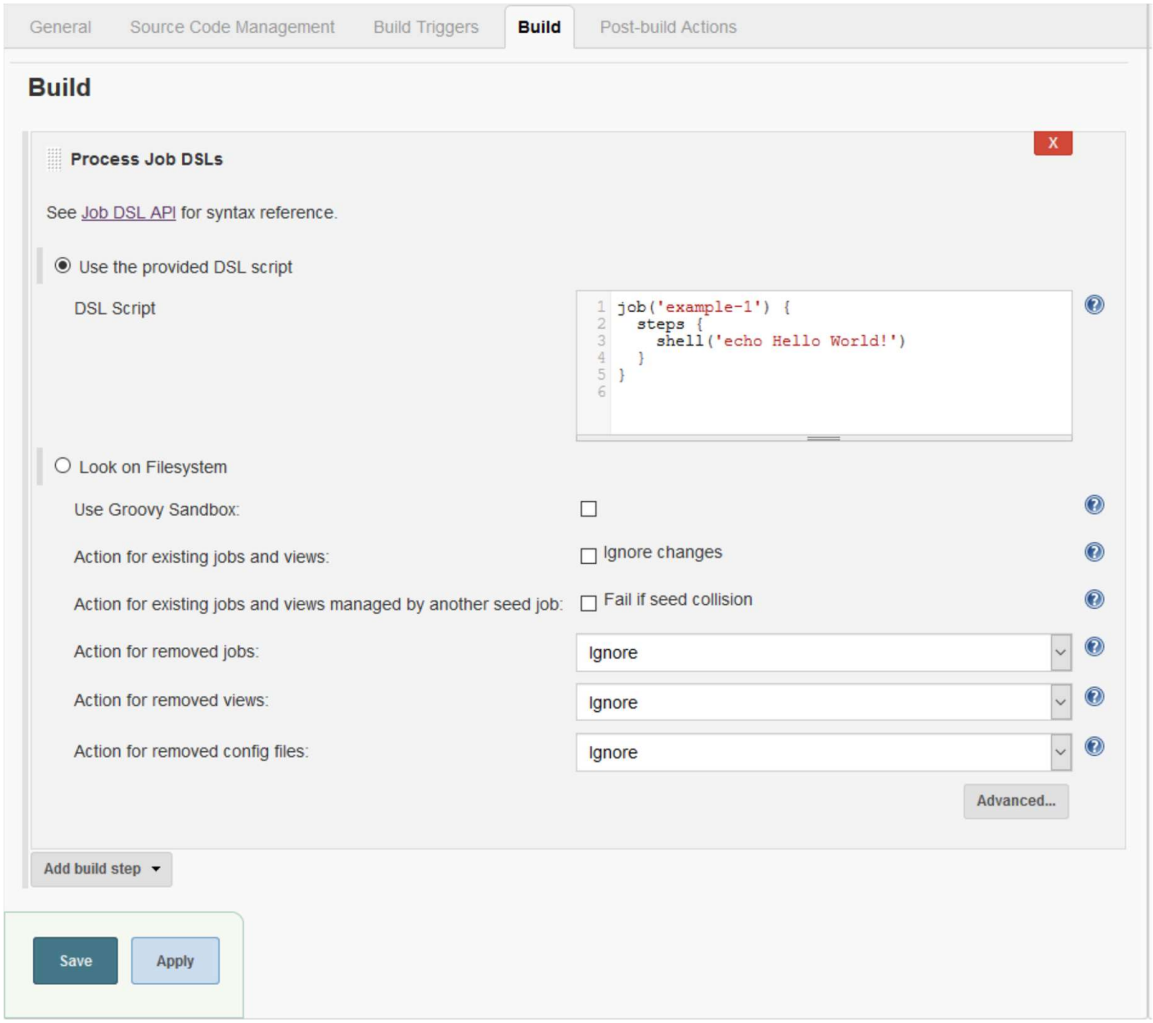
Then, create a new job and associate a new step “Process Job DSLs”:



Do the copy of the following code to the field “DSL Script”:

```
job('example') {
   steps {
     shell('echo Hello World!')
   }
}
```

The result should be similar to the presented above:



Try to run the job, It will give an approval error:



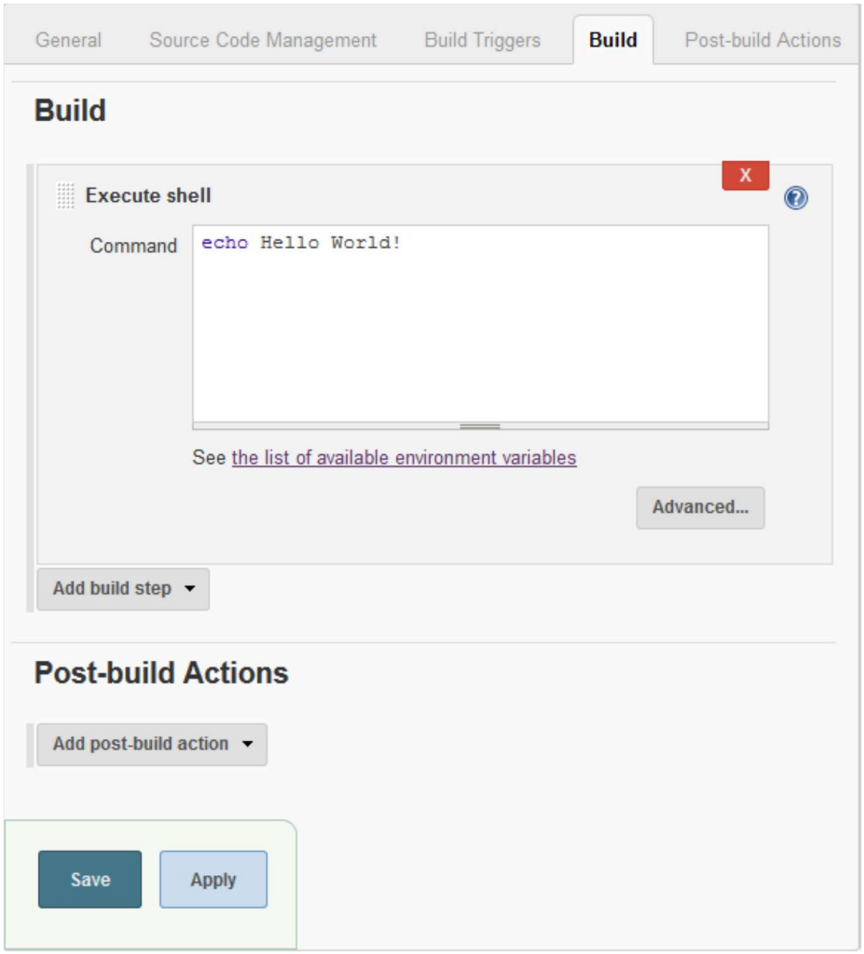
To fix the error go to “Manage Jenkins” and to section “in-process Script Approval”:



Try once again to run the build. Check that the job run with success and that the job was created with success according to the parameters of the DSL Job.

```
Started by user admin
Running as SYSTEM
Building in workspace /var/jenkins_home/workspace/seed
Processing provided DSL script
Added items:
    GeneratedJob{name='example'}
Finished: SUCCESS
```

Hence, the job DSL created the Job “example”. Check that the result is the one presented above:

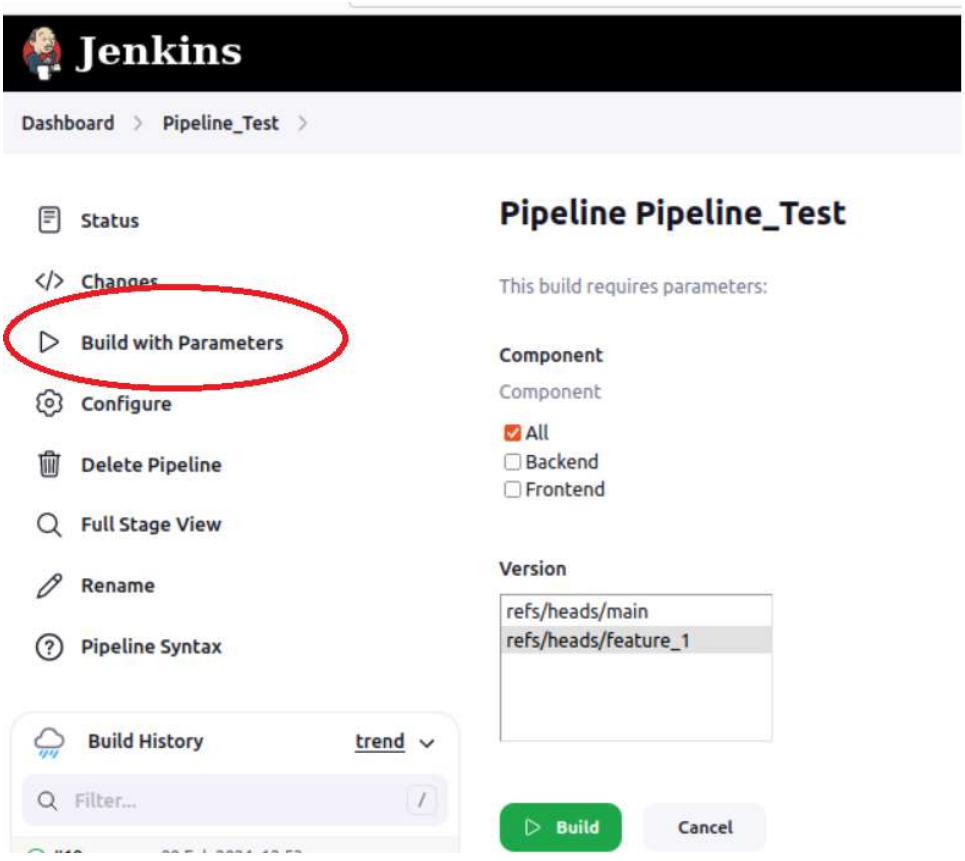

General
Source Code Management
Build Triggers
Build
Post-build Actions
Build
Execute shell
X
Command
echo Hello World!
See the list of available environment variables
Advanced...
Add build step
Post-build Actions
Add post-build action
Save
Apply

6. Creating a Jenkins Pipeline

We are going to create a pipeline using Declarative syntax. The requirements for the pipeline are:

- Steps in the pipeline to build and test the code
- Steps in the pipeline to build and push Docker images to Dockerhub
- Possibility to choose distinct branches to build the images (and distinct tags for each created image)
- Possibility to create multiple Docker images in a single pipeline.

When you press the "Build with Parameters" button, it should be possible to select the components eligible to build and push to DockerHub. Additionally, the student can choose what branch it wants to be triggered by Pipeline. You can check an example below:



The pipeline after finished one iteration will have a look similar to the below image:



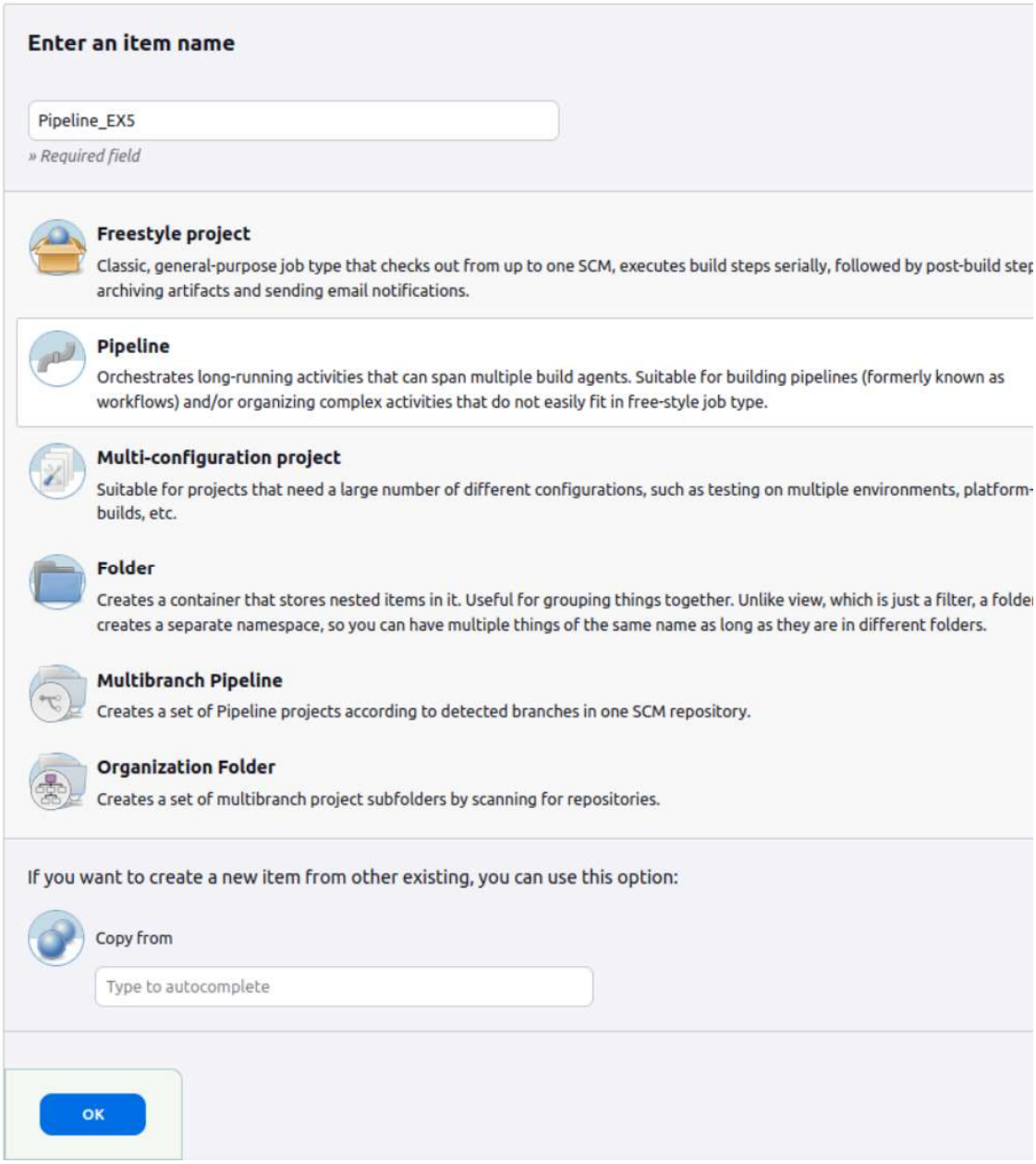
Try to create the pipeline, using the information from the slides and from previous exercises.

Resolution:

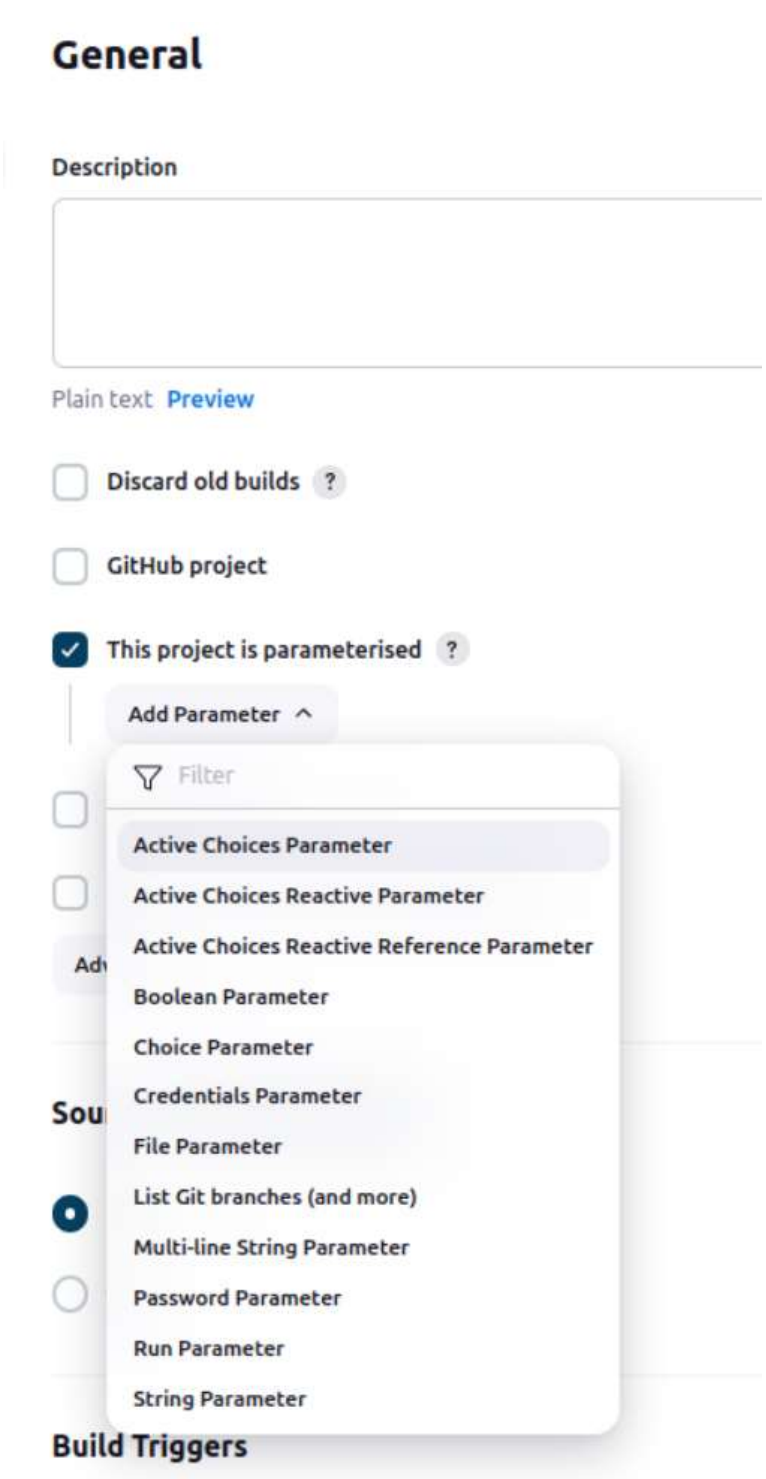
This pipeline uses two new plugins. The first is to list the components and the other to list the Git branches. Hence, start by installing the plugins needed by the Pipeline. To install the plugins:

- *List components:* From your Jenkins dashboard navigate to **Manage Jenkins > Manage Plugins** and select the **Available** tab. Locate this plugin by searching for *uno-choice*.
- *List Git Branches:* From your Jenkins dashboard navigate to **Manage Jenkins > Manage Plugins** and select the **Available** tab. Locate this plugin by searching for *list-git-branches-parameter*.

Afterwards, create a new Job (e.g. New Item)



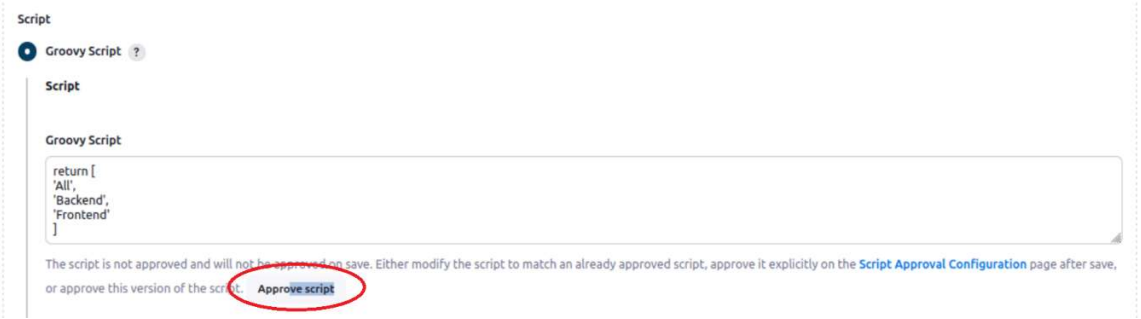
After pressing the button “OK”, the screen related with the Job Configuration will appear.
Start by checking the button “This project is parameterised” and Add Parameter “Active Choices Parameter”:



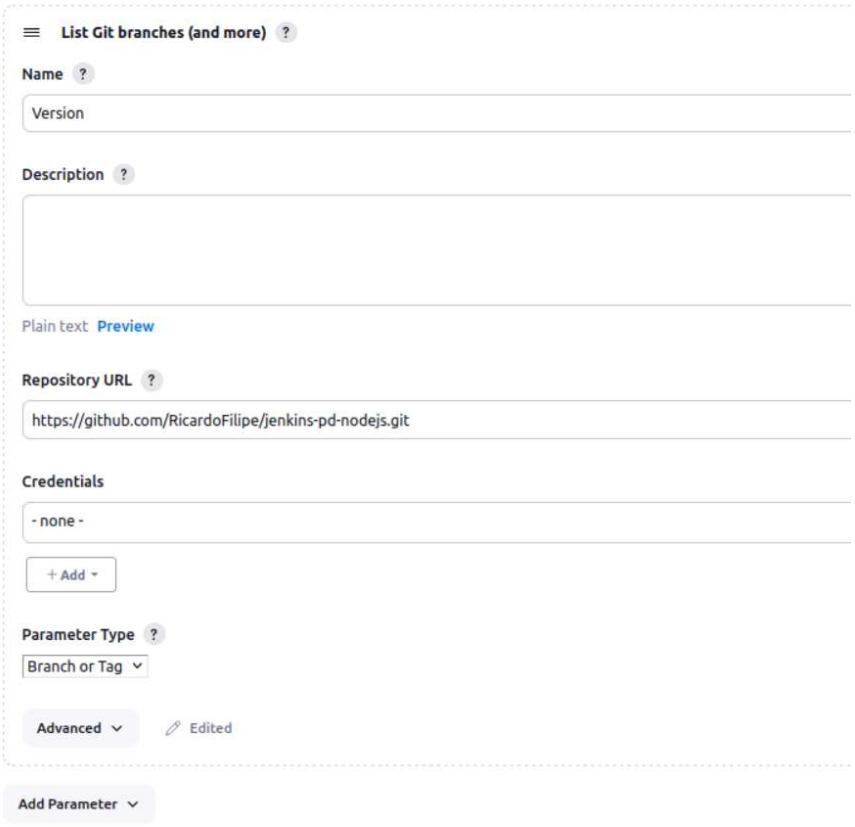
And a new tab will appear to configure.
You can put the following information:



You might have the following error presented in the next figure. In that case, approve the script:



Then, once again Add a new Parameter, but in this case a “List Git branches (and more)”.



You don't need to put credentials due to the fact that the Repository from the above image is public. However, if you want to use your own (private) repository, keep in mind that you need to change the Git authentication parameters.

The above steps were basically to configure the possibility to choose the components and Git Branch that you want to use on the pipeline.

Now, we are going to create the pipeline.
The next figure describes the overall configuration, including:

- The Git Repository
- The name where it is the Jenkins Pipeline
- Branch Identifier (we are using the alias Version previously created)
- Don't forget to uncheck "Lightweight checkout". It will be needed to ensure Jenkins read the variable $Version



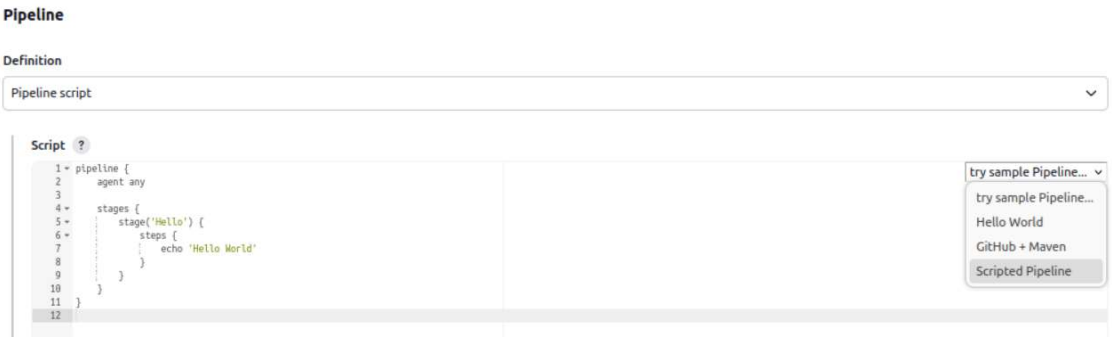
The content of the declarative pipeline is here (in the Jenkinsfile).
Try to understand the file content and run the pipeline. Change the pipeline to use your own Docker User and Github repository.

Change the Parameters (e.g. Choice Parameters or Boolean Parameters) to understand the distinct possibilities of Jenkins parameters.

Note: Although the stage 'Build and Publish BE' uses the same Dockerfile, you can select another filename to build a distinct part of the application and publish to a distinct Docker repository.

7. Run Jenkins Pipeline Samples

Run the Jenkins Pipeline Samples. Understand the distinct possibilities in terms of use and also language (declarative or scripted pipeline)



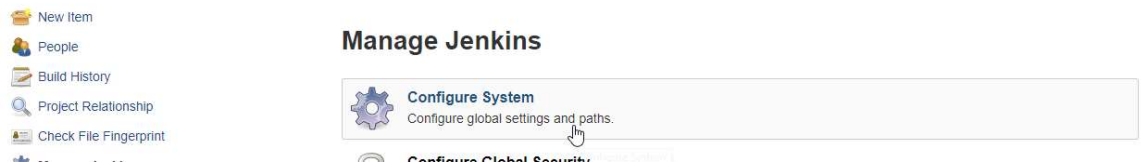
8. Email Integration

We are going to test the email notification through Jenkins pipeline. It is a very powerful tool to ensure that the development team is aware of some broken pipeline.

First, we need to add the email plugin to Jenkins:



Then, we are going to configure the email, through Configure System:

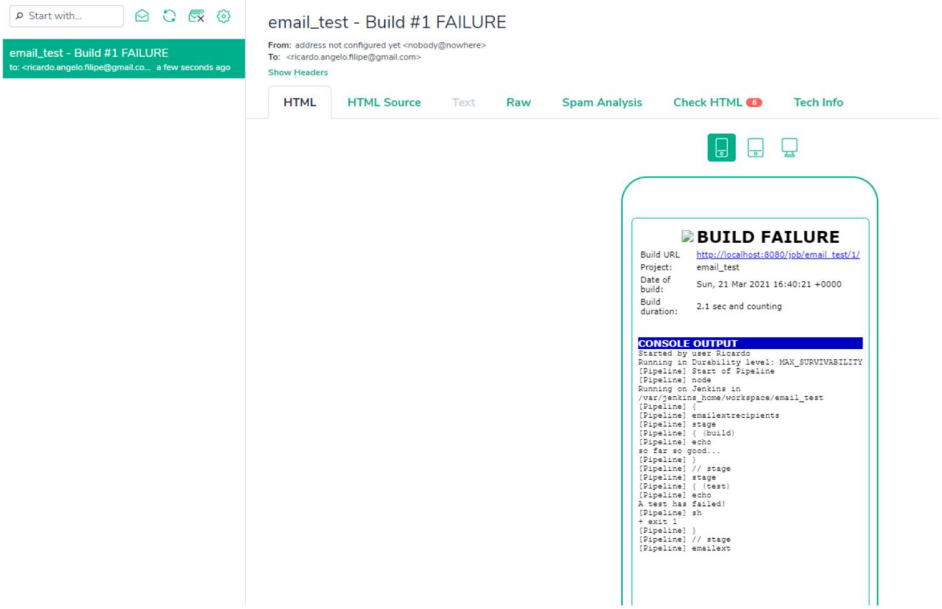


To send emails, the student can use mailtrap application, available in https://mailtrap.io/
After sign up in the platform you can use the credentials created by mailtrap.
E.g.



Regarding the pipeline, you can check the example available here

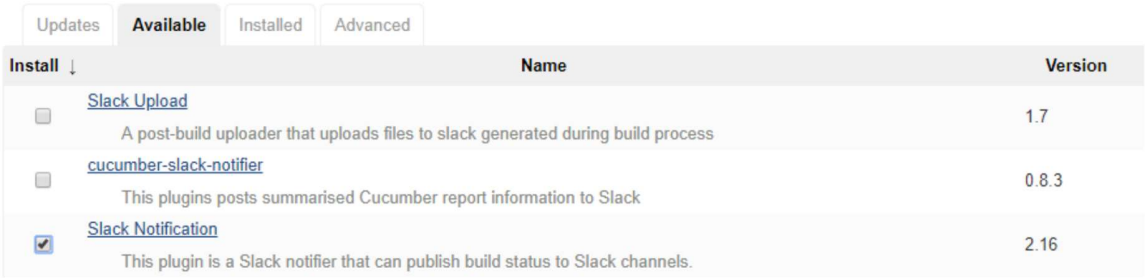
The output of the pipeline should be similar to the below image:



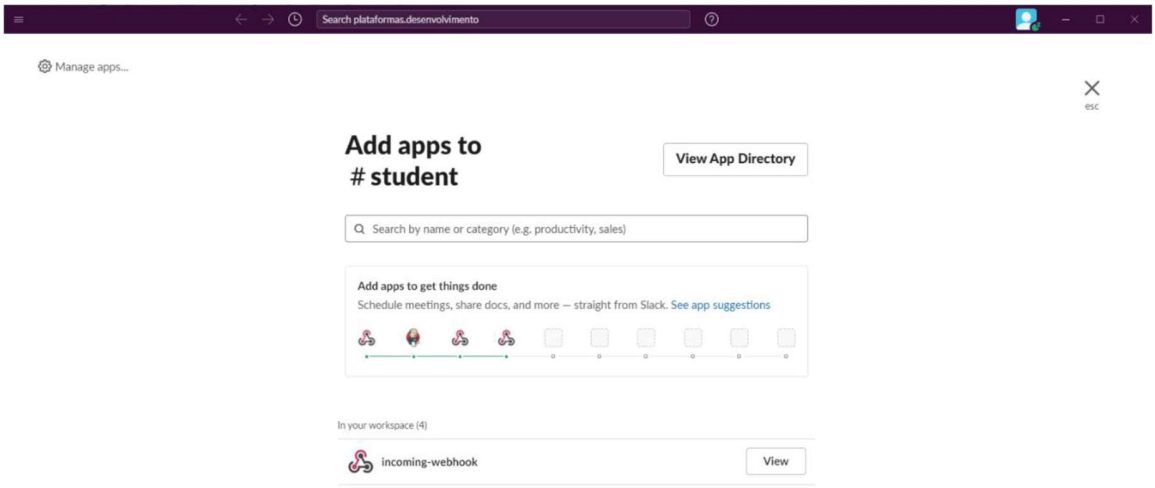
9. Slack Integration

Other useful tool is the possibility to send messages to a Slack channel. This ensure that Development (or other interested parts) are aware of the pipeline status.

First, we need to install the Slack plugin for Jenkins:



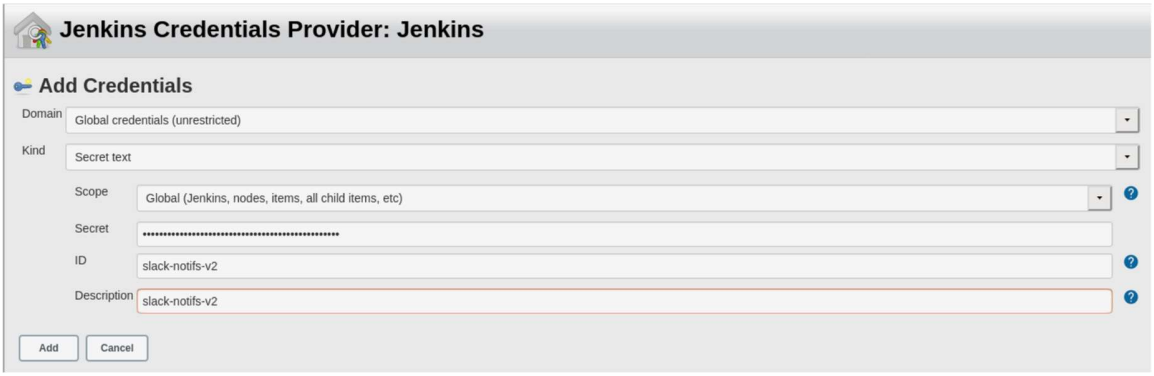
Then we will add the app "incoming webhooks" to the Slack channel.
For this, create the webhook on Slack:



Then configure the Slack plugin on Jenkins:

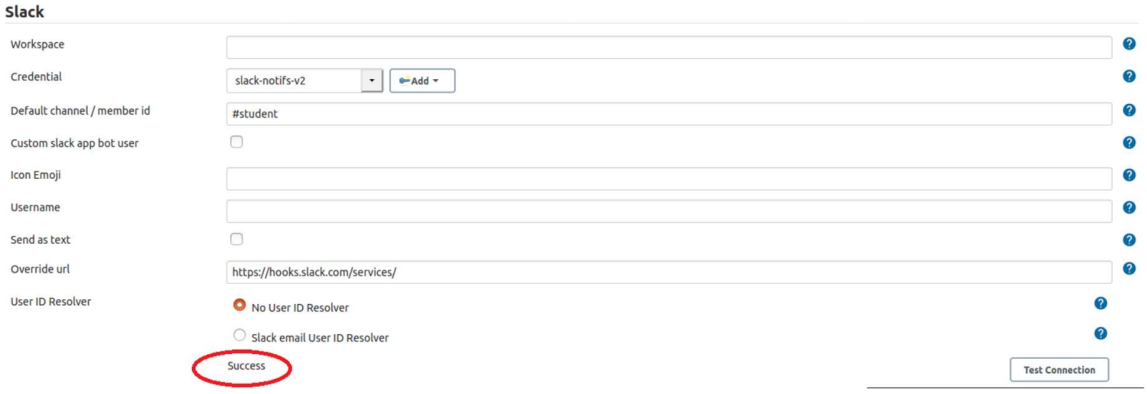


The credentials will have a look similar to the next image:

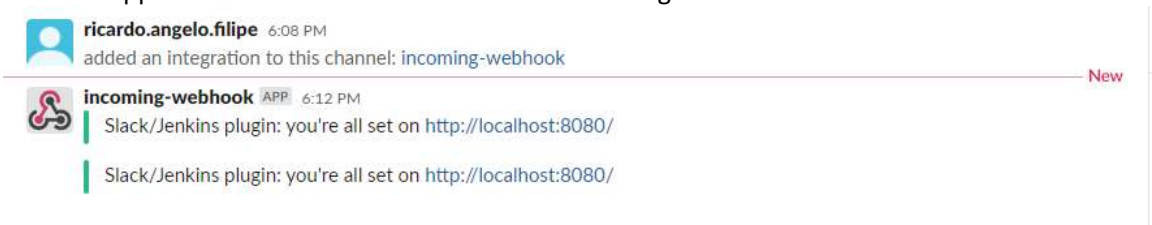


The secret matches the string in the webhook after "https://hooks.slack.com/services/". Id and Description can be chosen by the student.

To test the plugin notification you can send a dummy message:

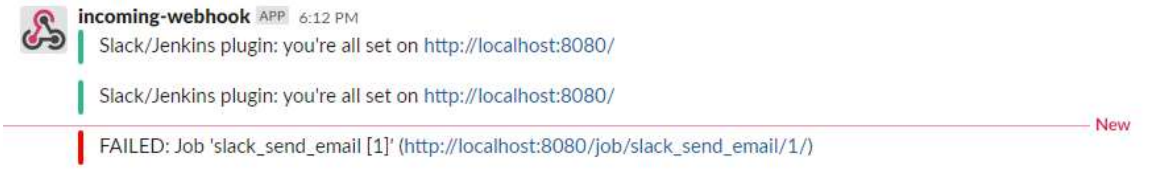


That will appear on Slack channel similar to the next image:



Regarding pipeline integration with Slack you can check an example pipeline here

The result will be similar to the next image:



To further information (e.g. change the default channel), please consult the official webpage